

ZESPÓŁ SZKÓŁ Nr 10

im. STANISŁAWA STASZICA

Temat: Aplikacja do planowania czytania

Poniższe sprawozdanie przedstawia szczegółowy opis stworzonej aplikacji do planowania czytania, obejmujący ogólny zarys projektu, zastosowane technologie, strukturę kodu oraz szczegółowe omówienie poszczególnych funkcji i metod wykorzystywanych w programie.

1.Wprowadzenie

Projekt “Aplikacja do planowania czytania” powstał z myślą o ułatwieniu użytkownikowi organizacji czytania książek, co przekłada się na większą liczbę osób, które czyta książki regularnie. Głównym celem aplikacji jest umożliwienie dodawania książek wraz z informacjami o całkowitej liczbie stron, liczbie dni przeznaczonych na przeczytanie oraz ocenę. Dodatkowo aplikacja pozwala na rozróżnianie książek fabularnych od niefabularnych, co umożliwia podanie dodatkowych informacji, takich jak gatunek literacki.

Aplikacja została napisana w języku Java i wykorzystuje podejście obiektowe. Kod został podzielony na trzy główne klasy:

- **Book** – klasa bazowa reprezentująca ogólny model książki.
- **FictionBook** – rozszerzenie klasy Book, umożliwiające uwzględnienie książek fabularnych (dodając informacje o gatunku)
- **ReadingPlan** - główna klasa odpowiedzialna za interakcje z użytkownikiem, zarządzanie książkami oraz operacje na plikach (zapisywanie i odczyt danych)

2.Technologie i narzędzia

Aplikacja wykorzystuje następujące technologie oraz narzędzia:

- **Język Java** – podstawowy język programowania wykorzystywane do implementacji aplikacji.
- **Programowanie Obiektowe (OOP)** - struktury klas, dziedziczenie i polimorfizm, dzięki czemu można łatwo zarządzać różnymi typami książek.
- **Obsługa wejścia/wyjścia (I/O)** - zastosowano klasy `BufferedReader`, `BufferedWriter` oraz `FileReader/FileWriter` do operacji na plikach. To pozwala na trwałe przechowywanie informacji o książkach w pliku tekstowym.
- **Biblioteka Scanner** - używana do pobierania danych wejściowych od użytkownika z konsoli.

- **Obsługa wyjątków (Exception Handling)** - mechanizmy zabezpieczające przed błędem odczytu, zapisu oraz błędami związanymi z konwersją danych.

Dzięki tym technologiom, aplikacja uzyskuje przejrzystą strukturę, łatwość rozbudowy oraz niezawodność w działaniu.

3. Struktura aplikacji

3.1 Klasa Book

Klasa Book reprezentuje podstawowy model książki i zawiera następujące elementy:

- **Pola:**
 - Title - tytuł książki
 - TotalPages – liczba stron książki
 - DaysToRead – liczba dni przeznaczonych na przeczytanie
 - Rating – ocena książki (zakres od 0 do 5; 0 oznacza brak oceny)
- **Konstruktor:** Konstruktor umożliwia inicjalizację wszystkich powyższych pól.
- **Metoda displayInfo():** Metoda wyświetla szczegółowe informacje dotyczące książki, w tym tytuł, liczbę stron, liczbę dni przeznaczonych na czytanie oraz oblicza przybliżoną liczbę stron do przeczytania dziennie (dzieląc całkowitą liczbę stron przez liczbę dni). Na końcu wywoływana jest metoda displayRating().
- **Metoda displayRating():** Metoda sprawdza, czy książka została oceniona (czy wartość rating jest większa od 0). Jeśli tak, wyświetla ocenę, w przeciwnym razie informuje, że jest brak oceny.

Dzięki takiej implementacji klasa Book stanowi podstawę dla prezentowania zarówno książek fabularnych, jak i niefabularnych.

3.2 Klasa FictionBook

Klasa FictionBook dziedziczy po klasie Book i dodaje jedno dodatkowe pole:

- **Pole:**
 - Genre – gatunek książki.
- **Konstruktor:** Konstruktor klasy FictionBook wywołuje konstruktor klasy bazowej Book przekazując podstawowe informacje o książce, a następnie inicjalizuje dodatkowe pole genre.
- **Metoda displayInfo():** Metoda ta została nadpisana (override), aby prócz informacji z klasy bazowej wyświetlić również gatunek książki. Dzięki temu rozróżnienie między książkami fabularnymi a niefabularnymi jest widoczne w interfejsie użytkownika.

Dzięki zastosowaniu dziedziczenia możliwe jest wprowadzenie specyficznych informacji dla książek fabularnych bez powielania kodu.

3.3 Klasa ReadingPlan

Główna klasa aplikacji, ReadingPlan, odpowiada za interakcję z użytkownikiem, zarządzanie listą książek oraz przeprowadzenie operacji odczytu i zapisu do pliku. Jej główne elementy to:

- **Pola:**
 - Books – dynamiczna lista przechowująca obiekty typu Book (w tym również instancje FictionBook dzięki polimorfizmowi),
 - Scanner – obiekty klasy scanner do pobierania danych użytkownika,
 - FILE_NAME - stała zawierająca nazwę pliku (books.txt), w którym są zapisywane informacje o książkach.
- **Konstruktor:** konstruktor inicjalizuje listę books, tworzy nowy obiekt Scanner i wywołuje metodę loadBooksFile(), która wczytuje wcześniej zapisane dane z pliku.
- **Metoda run():** To główna pętla aplikacji. Prezentuje użytkownikowi menu, pobiera wybór opcji (dodanie książki, wyświetlenie książek lub zakończenie działania) i wykonuje odpowiednią akcję na podstawie wyboru użytkownika. W momencie wyboru opcji zakończenia, zapisywane są dane do pliku oraz zamykane jest obiekt Scanner.
- **Metoda displayMenu():** Odpowiada za wyświetlanie menu z dostępnymi opcjami, co czyni interakcję z użytkownikiem bardziej przejrzystą i intuicyjną.
- **Metoda addBook:**

Ta metoda umożliwia użytkownikowi dodawanie nowych książek do listy:

- Najpierw pobierane są podstawowe informacje takie jak tytuł, liczba stron, liczba dni przeznaczonych na czytanie oraz ocena.
- Następnie użytkownik jest zapytany o to, czy książka jest fabularna. Jeśli odpowiedź to "Tak", program prosi o dodatkową informację - gatunek książki - i tworzy obiekt FictionBook.
- W przeciwnym wypadku tworzony jest obiekt klasy Book.
- Na końcu potwierdzone jest dodanie książki.
- **Metoda displayBooks():** Metoda ta iteruje po liście books i wywołuje metodę displayInfo() każdego dodanego obiektu, umożliwiając w ten sposób prezentację pełnych informacji o książkach zapisanych przez użytkownika. W przypadku pustej listy informuje użytkownika, że nie dodano jeszcze żadnych książek.
- **Metoda exit():** Ta metoda jest odpowiedzialna za zakończenie działania aplikacji. Przed wyjściem wykonywane jest zapisanie danych książek do pliku

(poprzez wywołanie metody `saveBooksToFile()`), po czym następuje komunikat potwierdzający zapis i zamknięcie skanera.

- **Metoda `saveBooksToFile()`:**

Metoda odpowiada za zapisanie stanu listy książek do pliku tekstowego. Każda książka jest zapisywana w formacie:

- Dla książek fabularnych F ; tytuł ; liczba stron; liczba dni; ocena ; gatunek
- Dla książek niefabularnych N; tytuł; liczba stron ; liczba dni; ocena

- **Metoda `loadBooksFromFile()`:** Ta metoda sprawdza, czy plik o nazwie `books.txt` istnieje. Jeśli tak, odczytuje każdą linię, parsuje jej zawartość i w zależności od oznaczenia (F lub N) tworzy obiekty `FictionBook` lub `Book`. Dzięki temu aplikacja jest w stanie przywrócić poprzedni stan danych po ponownym uruchomieniu.

4. Podsumowanie

Aplikacja „Plan czytania” jest przykładem dobrze zorganizowanego projektu opartego na zasadach programowania obiektowego w Javie. Dzięki klarownej separacji odpowiedzialności między klasami - `Book` jako podstawowa klasa danych, `FictionBook` rozszerzająca bazowy model o dodatkowe informacje, oraz `Reading Plan` zarządzająca interakcją z użytkownikiem i operacjami na plikach - projekt zapewnia łatwość utrzymania oraz możliwość rozbudowy.

Zastosowane technologie, takie jak operacje na plikach, obsługa wyjątków oraz interakcja z użytkownikiem za pomocą konsoli, gwarantują funkcjonalność aplikacji w zakresie trwałego przechowywania danych i wygodnej obsługi. Możliwość rozróżnienia książek fabularnych od niefabularnych pozwala na bardziej szczegółowe zarządzanie informacjami i podnosi użyteczność aplikacji.

Projekt stanowi solidną podstawę dla dalszych modyfikacji - na przykład można rozszerzyć funkcjonalność o możliwość edycji lub usuwania książek, dodać interfejs graficzny czy rozbudować system oceniania. Dzięki przemyślanej strukturze kodu i zastosowaniu zasad programowania obiektowego, przyszłe rozbudowy i modyfikacje będą stosunkowo łatwe do wdrożenia.

Powyższe sprawozdanie stanowi kompleksowy opis stworzonej aplikacji, ukazując zarówno jej strukturę, wykorzystane technologie, jak i szczegółowy opis poszczególnych funkcji. Dzięki temu użytkownik lub oceniający projekt ma pełny obraz działania i możliwości aplikacji do planowania czytania.

Filip Rutkowski

Klasa 2ap

WARSZAWA 14.05.2025